Strategy-Pattern

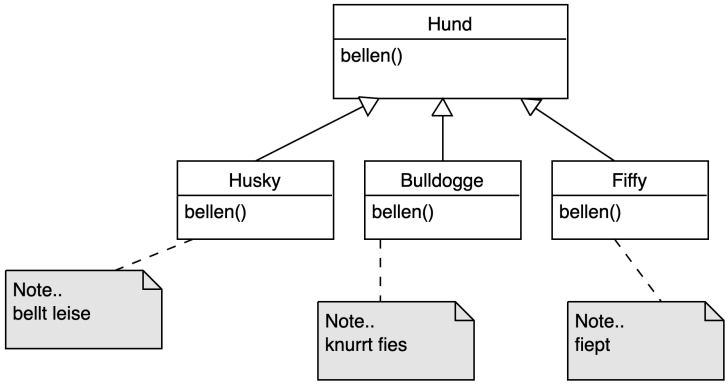
Carsten Gips (HSBI)

TL;DR
Das Verhalten von Klassen kann über Vererbungshierarchien weitergegeben und durch Überschreiben in den erbenden Klassen verändert werden. Dies führt häufig schnell zu breiten und tiefen Vererbungsstrukturen.
Das Strategy-Pattern ist ein Entwurfsmuster, in dem Verhalten stattdessen an passende Klassen/Objekte ausgelagert (delegiert) wird.
Es wird eine Schnittstelle benötigt (Interface oder abstrakte Klasse), in dem Methoden zum Abrufen des gewünschten Verhaltens definiert werden. Konkrete Klassen leiten davon ab und implementieren das gewünschte konkrete Verhalten.
In den nutzenden Klassen wird zur Laufzeit eine passende Instanz der (Strategie-) Klassen übergeben (Konstruktor, Setter, …) und beispielsweise über ein Attribut referenziert. Das gewünschte Verhalten muss nun nicht mehr in der nutzenden Klasse selbst implementiert werden, stattdessen wird einfach auf dem übergebenen Objekt die Methode aus der Schnittstelle aufgerufen. Dies nennt man auch "Delegation", weil die Aufgabe (das Verhalten) an ein anderes Objekt (hier das Strategie-Objekt) weiter gereicht (delegiert) wurde.
Videos

- VL Strategy-Pattern
- Demo Strategy-Pattern

---

1 Wie kann man das Verhalten einer Klasse dynamisch ändern?

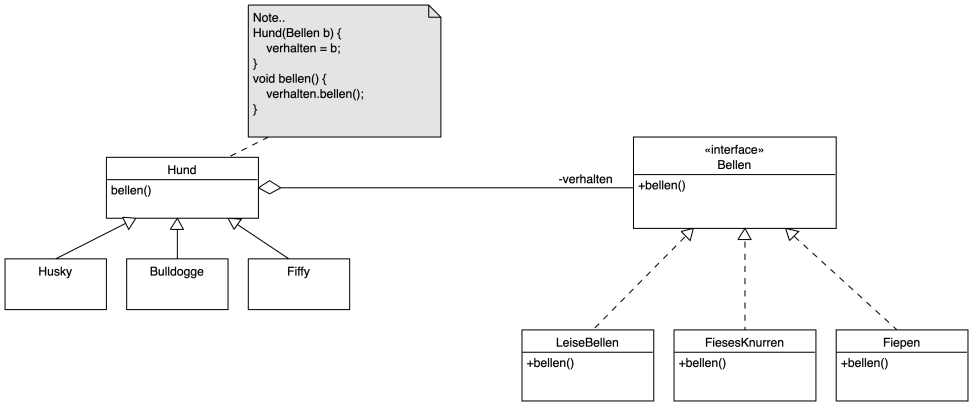


Modellierung unterschiedlicher Hunderassen: Jede Art bellt anders.

Es bietet sich an, die Hunderassen von einer gemeinsamen Basisklasse `Hund` abzuleiten, um die Hundeartigkeit allgemein sicherzustellen.

Da jede Rasse anders bellen soll, muss jedes Mal die Methode `bellen` überschrieben werden. Das ist relativ aufwändig und fehleranfällig. Außerdem kann man damit nicht modellieren, dass es beispielsweise auch konkrete Bulldoggen geben mag, die nur leise fiepen …

2 Lösung: Delegation der Aufgabe an geeignetes Objekt



Der Hund delegiert das Verhalten beim Bellen an ein Objekt, welches beispielsweise bei der Instantiierung der Klasse übergeben wurde (oder später über einen Setter). D.h. die Methode `Hund#bellen` bellt nicht mehr selbst, sondern ruft auf einem passenden Objekt eine vereinbarte Methode auf.

Dieses passende Objekt ist hier im Beispiel vom Typ `Bellen` und hat eine Methode `bellen` (Interface). Die verschiedenen Bell-Arten kann man über eigene Klassen implementieren, die das Interface einhalten.

Damit braucht man in den Klassen für die Hunderassen die Methode `bellen` nicht jeweils neu überschreiben, sondern muss nur bei der Instantiierung eines Hundes ein passendes `Bellen`-Objekt mitgeben.

Als netten Nebeneffekt kann man so auch leicht eine konkrete Bulldogge realisieren, die eben nicht fies knurrt, sondern leise fiept …

Entwurfsmuster: **Strategy Pattern**

3 Exkurs UML: Assoziation vs. Aggregation vs. Komposition

Eine **Assoziation** beschreibt eine Beziehung zwischen zwei (oder mehr) UML-Elementen (etwa Klassen oder Interfaces).

Eine **Aggregation** (leere Raute) ist eine Assoziation, die eine *Teil-Ganzes-Beziehung* hervorhebt. Teile können dabei ohne das Ganze existieren (Beispiel: Personen als Partner in einer Ehe-Beziehung). D.h. auf der einbindenden Seite (mit der leeren Raute) hat man implizit `0..*` stehen.

Eine **Komposition** (volle Raute) ist eine Assoziation, die eine Teil-Ganzes-Beziehung hervorhebt. Teile können aber nicht ohne das Ganze existieren (Beispiel: Gebäude und Stockwerke: Ein Gebäude besteht aus Stockwerken, die ohne das Gebäude aber nicht existieren.). D.h. auf der einbindenden Seite (mit der vollen Raute) steht implizit eine `1` (ein Stockwerk gehört genau zu einem Gebäude, ein Gebäude besteht aber aus mehreren Stockwerken).

Siehe auch Aggregation, Assoziation und Klassendiagramm.

4 Zweites Beispiel: Sortieren einer Liste von Studis

Sortieren einer Liste von Studis: `Collections.sort` kann eine Liste nach einem Default-Kriterium sortieren oder aber über einen extra `Comparator` nach benutzerdefinierten Kriterien … Das Verhalten der Sortiermethode wird also quasi an dieses Comparator-Objekt delegiert …

```
public class Studi {
    private String name;
    public Studi(String name) { this.name = name; }

    public static void main(String[] args) {
        List<Studi> list = new ArrayList<Studi>();
        list.add(new Studi("Klaas"));
        list.add(new Studi("Hein"));
        list.add(new Studi("Pit"));

        // Sortieren der Liste (Standard-Reihenfolge)?!
        // Sortieren der Liste (eigene Reihenfolge)?!
    }
}
```

Konsole strategy.SortDefault, strategy.SortOwnCrit

Anmerkung: Die Interfaces `Comparable` und `Comparator` und deren Nutzung wurde(n) in OOP besprochen. Anonyme Klassen wurden ebenfalls in OOP besprochen. Bitte lesen Sie dies noch einmal in der Semesterliteratur nach, wenn Sie hier unsicher sind!

5 Hands-On: Strategie-Muster

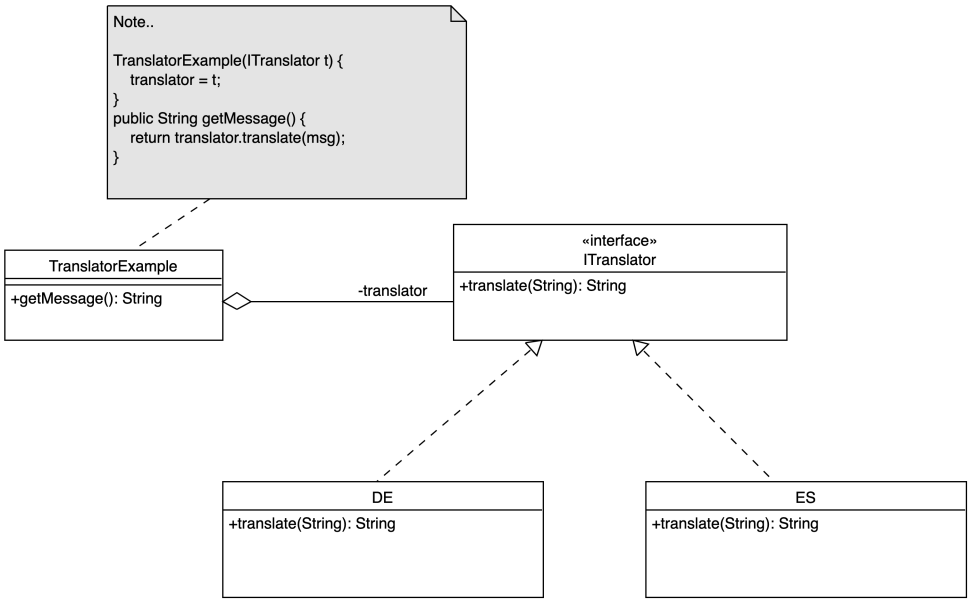
Implementieren Sie das Strategie-Muster für eine Übersetzungsfunktion:

- Eine Klasse liefert eine Nachricht (`String`) mit `getMessage()` zurück.
- Diese Nachricht ist in der Klasse in Englisch implementiert.
- Ein passendes Übersetzerobjekt soll die Nachricht beim Aufruf der Methode `getMessage()` in die Ziel-Sprache übersetzen.

Fragen:

1. Wie muss das Pattern angepasst werden?
2. Wie sieht die Implementierung aus?

6 Auflösung



Konsole strategy.TranslatorExample

7 Wrap-Up

Strategy-Pattern: Verhaltensänderung durch Delegation an passendes Objekt

- Interface oder abstrakte Klasse als Schnittstelle
- Konkrete Klassen implementieren Schnittstelle => konkrete Strategien
- Zur Laufzeit Instanz dieser Klassen übergeben (Aggregation) …
- … und nutzen (Delegation)

8 Zum Nachlesen

- Eilebrecht und Starke (2013)
- Gamma u. a. (2011)
- Kleuker (2018)

Lernziele

- k3: Strategie-Entwurfsmuster praktisch anwenden

Quizzes

- Quiz Strategy-Pattern (ILIAS)

Challenges

Implementieren Sie das Spiel “Schere,Stein,Papier” (Spielregeln vergleiche wikipedia.org/wiki/Schere,Stein,Papier) in Java.

Nutzen Sie das Strategy-Pattern, um den Spielerinstanzen zur Laufzeit eine konkrete Spielstrategie mitzugeben, nach denen die Spieler ihre Züge *berechnen*. Implementieren Sie mindestens drei unterschiedliche konkrete Strategien.

Hinweis: Eine mögliche Strategie könnte sein, den Nutzer via Tastatureingabe nach dem nächsten Zug zu fragen.

Gehen Sie bei der Lösung der Aufgabe methodisch vor:

1. Stellen Sie sich eine Liste mit relevanten Anforderungen zusammen.
2. Erstellen Sie (von Hand) ein Modell (UML-Klassendiagramm):
    - Welche Klassen und Interfaces werden benötigt?
    - Welche Aufgaben sollen die Klassen haben?
    - Welche Attribute und Methoden sind nötig?
    - Wie sollen die Klassen interagieren, wer hängt von wem ab?
3. Implementieren Sie Ihr Modell in Java. Schreiben Sie ein Hauptprogramm, welches das Spiel startet, die Spieler ziehen lässt und dann das Ergebnis ausgibt.
4. Überlegen Sie, wie Sie Ihr Programm sinnvoll manuell testen können und tun Sie das.

Quellen

Eilebrecht, K., und G. Starke. 2013. *Patterns kompakt*. Springer.

Gamma, E., R. Helm, R. E. Johnson, und J. Vlissides. 2011. *Design Patterns*. Addison-Wesley.

Kleuker, S. 2018. *Grundkurs Software-Engineering mit UML*. Springer Vieweg. https://doi.org/10.1007/978-3-658-19969-2.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 71232c0 (tooling: shift headings (use h1 as top-level headings), 2025-04-29)